

agentr: Practical Scaffolding for Coding-AI Workflows

Oliver Zhou

Lily Li

Abstract

`agentr` is an R package for scaffolding coding-AI workflows through inspectable task specifications, review artifacts, and autonomy boundaries. Its contribution is not to provide another model wrapper, chat interface, or autonomous execution engine. Instead, it supplies a project-local structure through which external coding AIs—including repository-aware assistants, command-line agents, and GUI/API-backed model tools—can help design and implement work while leaving intermediate decisions visible. The package focuses on three related capabilities. First, it turns workflow design, memory, curated knowledge, interface expectations, and implementation notes into editable files. Second, it supports a coding-assistant workflow in which these files can be inferred, validated, reviewed, revised, implemented, and versioned with Git. Third, it renders specifications into visual review pages that help humans inspect decomposition, assumptions, gates, and warnings before downstream execution. These capabilities position `agentr` as a practical middle layer between ordinary prompting and full agent runtimes, especially for R-centered, repository-based work where AI assistance is useful but intermediate reasoning, provenance, and human authority must remain inspectable.

Introduction

AI-assisted coding increasingly sits between two unsatisfactory extremes. At one extreme, a developer asks a model for a final artifact—a script, analysis, report, patch, or visualization—and receives an answer whose intermediate assumptions are mostly embedded in a chat transcript. At the other extreme, an autonomous agent framework may plan, edit files, call tools, and coordinate steps, but the workflow design can become difficult for a human reviewer to understand or constrain once execution begins. The practical problem is not simply whether the model is capable. It is whether the work can be decomposed, reviewed, corrected, and reused before a probabilistic assistant turns an instruction into code.

This problem is becoming more important because agentic AI systems are increasingly described as combinations of planning, memory, tool use, feedback, and action rather than as single-turn text generators (Sapkota, Roumeliotis, and Karkee 2025; Wissuchek and Zschech 2025; Yang et al. 2025). In software engineering, that shift changes the unit of concern. The question is no longer only whether a model can produce code, but whether the surrounding workflow helps the model acquire the right context, preserve human control, validate intermediate decisions, and leave evidence for later review. Studies and frameworks for AI coding assistance repeatedly point to this workflow-level condition: adoption varies by task stage and perceived control (Sergeyuk et al. 2024), repository-aware grounding can improve contextual alignment but does not

remove hallucination or retrieval-to-action gaps (Borghoff, Minas, and Schopp 2025), and human-in-the-loop software agents still require staged review because generated plans and code remain incomplete or context-sensitive (Takerngsaksiri et al. 2024). Comparative work on AI and human workflows reinforces the same point from another direction: agents may follow broadly similar high-level task structures, but they often rely on programmatic methods, struggle with visual or interface-heavy work, and require human judgment for verification-heavy steps (Z. Z. Wang et al. 2025).

`agentr` responds to this workflow-level problem. It treats intermediate task design as a first-class development artifact rather than as an implicit by-product of prompting. A task can be represented through editable specifications for workflow nodes and edges, review gates, memory fields, curated knowledge, interface expectations, autonomy stage, implementation targets, and review outputs. External coding AIs can still do substantial work, but their work is organized around project-local files that can be inspected, validated, revised, rendered, and committed to Git.

The paper therefore does not discuss every package feature with equal weight. It focuses on the three features that are most significant in relation to current agentic-AI and coding-assistant literature. The first is the standardized specification layer, which turns plans, assumptions, memory, and knowledge into portable artifacts. The second is the coding-assistant scaffolding pattern, which uses those artifacts to coordinate external AI assistance with human review and version control. The third is visual review, which makes specifications inspectable enough to support governance decisions about autonomy, risk, and implementation readiness. These features define the package’s main claim: `agentr` is not an agent runtime; it is scaffolding for making coding-AI-driven work structured, inspectable, and portable.

Figure 1 summarizes this boundary claim: `agentr` does not replace either human prompting or downstream agent execution, but supplies the reviewable project-local layer between them.

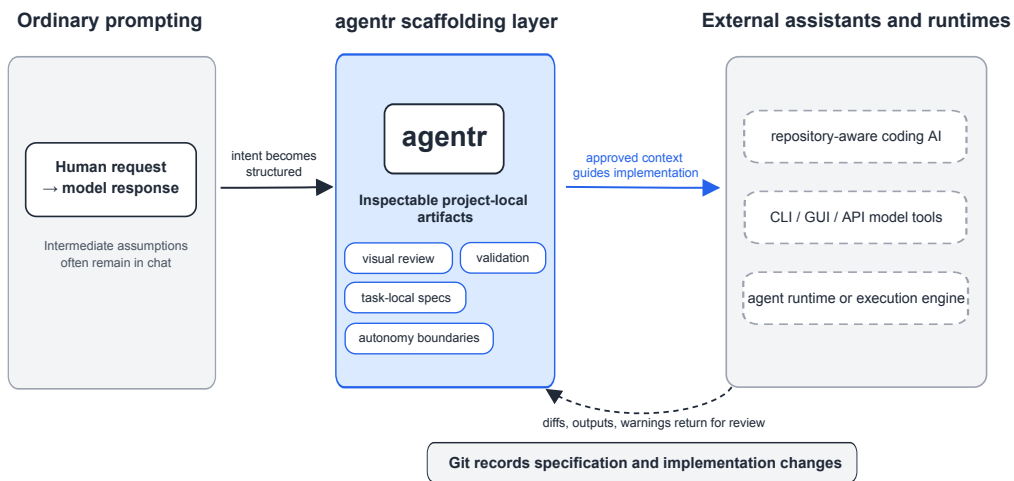


Figure 1: `agentr` as a middle layer between ordinary prompting and full agent runtimes.

This positioning matters because the hardest problem addressed by `agentr` is not model access, but the absence of reliable intermediate structure between intent and implementation.

The core problem: capable coding AIs without reliable intermediate structure

Modern coding assistants can inspect repositories, propose plans, edit files, run checks, and revise implementations. This capability creates a practical tension. Delegation becomes more attractive because the assistant can perform more of the mechanical work. At the same time, direct delegation gives the user limited control over the decisions that precede the final artifact. A completed patch may not reveal which assumptions were accepted, which files were considered, which constraints were ignored, or where human approval should have been required.

The tension is clearest in analytical, scientific, financial, policy, and technical-writing workflows. In these settings, the final output is valuable only if the path to it can be inspected. A chart is not sufficient unless the data source, transformations, exclusions, and interpretation are reviewable. A report is not sufficient unless the assumptions, heuristics, and citation choices can be checked. A software patch is not sufficient unless the design it implements was approved and the implementation can be compared against that design. The missing capability is not merely better generation. It is a durable intermediate structure that separates intent, design, evidence, implementation, and review.

This distinction matters because agentic workflows distribute error across more places than a single model response. Planning errors can affect decomposition. Retrieval errors can distort context. Memory errors can preserve stale or false information. Tool-use errors can modify files or environments. Human-approval errors can authorize work before assumptions are clear. Surveys and taxonomies of agentic systems describe memory, tool use, reasoning, interactivity, and self-improvement as important dimensions of agentic capability, but the same dimensions create governance and evaluation problems when they are not represented explicitly ([Wissuchek and Zschech 2025](#); [Du 2026](#); [Bansod 2025](#)). For coding work, this means that a system can appear productive while making it hard to reconstruct why a change happened or whether the right human judgment was applied at the right stage.

`agentr` addresses this problem by inserting a reviewable layer between human intent and downstream execution. Instead of asking a coding AI to move directly from instruction to final output, `agentr` asks the assistant to work through explicit artifacts. These artifacts provide more structural control than ordinary prompting, more durable provenance than a chat-only workflow, and more flexibility than a provider-specific agent framework. The relevant contrast is not that files are more formal than chat. The relevant contrast is functional: chat is a weak medium for maintaining approved workflow state, while project-local specifications can be validated, rendered, diffed, revised, and reused.

Contribution 1: task-local specifications as portable workflow artifacts

The first contribution of `agentr` is to standardize the intermediate artifacts that a coding-AI workflow needs before implementation. A workflow artifact records the task sequence, dependencies, gates, and implementation responsibilities. A memory artifact states what context should persist across interactions, including semantic, episodic, procedural, or task-specific memory. A knowledge artifact records curated domain content: concepts, rules, exceptions, heuristics, style expectations, evaluation criteria, risks, and assumptions. This matters because domain grounding is not only a retrieval problem: recent work on codified expert

knowledge suggests that explicit procedural rules and tacit expert principles can change the quality of agent outputs in ways that ordinary LLM+RAG assistance may not capture (Uulu et al. 2026). Interface and autonomy fields clarify what files, tools, APIs, or human decisions the assistant may touch. Review artifacts then render these specifications into a form that can be inspected by humans.

This standardization is intentionally partial. `agentr` does not try to define every possible runtime behavior of an autonomous agent. Instead, it standardizes the portion of the workflow that must remain portable across coding assistants and execution environments: what the task is, how it is decomposed, what knowledge supports it, what context should persist, what authority the assistant has, and what evidence remains for review. That scope is narrower than a full agent platform, but it directly addresses the bottleneck in repository-based AI assistance: the need to keep task design visible while implementation tools continue to change.

The design aligns with recent work on declarative and portable agent representations. Open-agent-specification approaches argue that agent workflows benefit from shared representations that separate design from runtime adapters (Amini et al. 2025). Declarative framework proposals similarly distinguish portable cognitive blueprints from platform-specific runtime engines, with explicit contracts around memory, tools, outputs, and safety (Cao et al. 2026). `agentr` adopts a smaller, practical version of the same separation for R-centered projects. Its specifications do not make behavior provider-independent in a strong runtime sense, but they make the task description, review expectations, and implementation boundaries independent of any single chat interface.

The cognitive value of these artifacts is that they externalize what would otherwise remain implicit. Workflow structure externalizes planning. Memory schema externalizes what should persist. Knowledge schema externalizes assumptions and evaluation criteria. Interface fields externalize what tools and files the assistant may use. Autonomy labels externalize who has decision authority at each stage. Review artifacts externalize the evidence a human needs before allowing implementation to proceed. The mechanism is therefore not abstraction for its own sake. It is the conversion of fragile prompt context into durable project context.

Figure 2 shows this specification layer as a stack of task-local artifacts. The important point is not that every task must use every field, but that the fields separate planning, context, authority, implementation, and review into inspectable units.

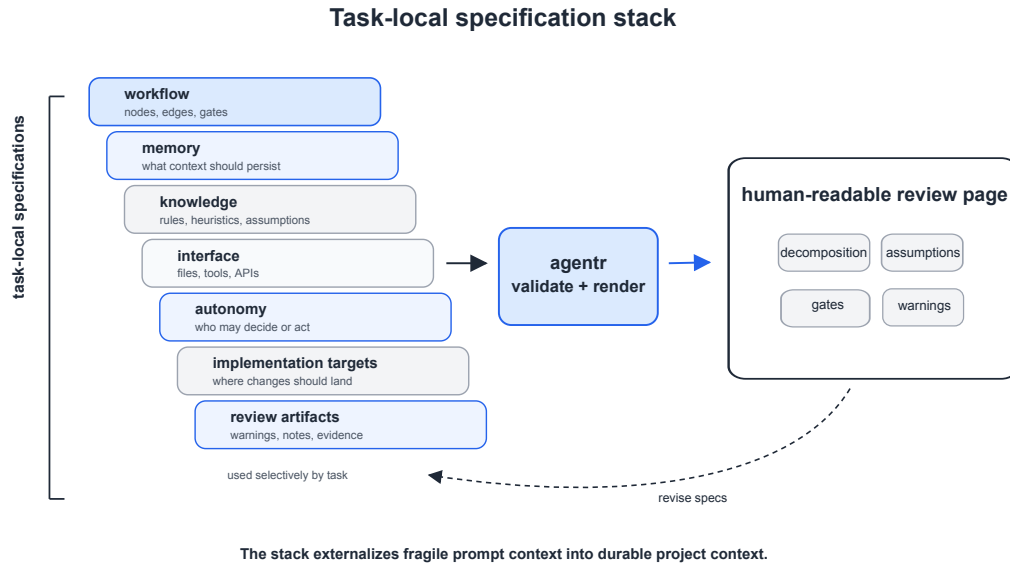


Figure 2: Task-local specification stack used to externalize workflow design, context, boundaries, and review evidence.

This separation explains why the specification layer can remain useful even when the external coding assistant, model provider, or execution environment changes.

Contribution 2: coding-assistant scaffolding around external AI and Git

The second contribution is a practical workflow for using external coding AIs without making `agentr` itself the autonomous execution engine. A typical use pattern is:

A typical use pattern is summarized in Figure 3. The important feature of the loop is that the assistant can help infer, revise, and implement, but approval and redirection remain attached to reviewable artifacts rather than hidden inside a chat transcript.

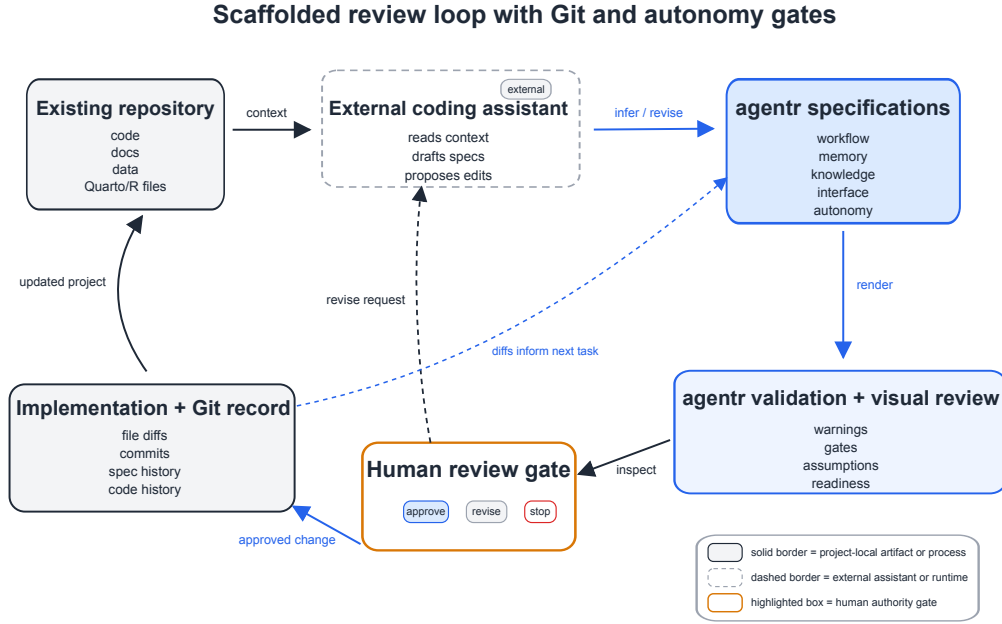


Figure 3: Scaffolded coding-AI review loop linking external assistants, agentr specifications, human review gates, and Git records.

This loop also explains why Git is not merely a storage mechanism in agentr workflows. It records how the design artifacts and the implementation evolve together, allowing reviewers to compare a proposed code change against the specification that authorized it.

This pattern treats coding AIs as powerful collaborators rather than invisible decision-makers. The human does not need to hand-write every line of implementation code. The assistant can inspect files, draft specifications, propose revisions, and implement approved changes. But the assistant’s work is routed through artifacts that make the task design visible: workflow graphs, memory schemas, knowledge items, review notes, warnings, implementation targets, and file diffs.¹

This scaffolding is useful because coding-AI performance depends on more than model capability. Developers are more cautious when tasks require project-specific context, high precision, and direct control over code changes (Sergeyuk et al. 2024). Repository-aware assistants can improve contextual alignment, but retrieval alone does not ensure correct action, because a model may still misuse tools, hallucinate, or fail to translate retrieved context into the needed code change (Borghoff, Minas, and Schopp 2025). Human-centered evaluations likewise suggest that model assistance is more reliable when humans lead the interaction, provide context, and validate outputs rather than over-delegating the entire development task (W. Wang et al. 2024). agentr operationalizes this lesson by making context, scope, and approval status part of the repository rather than part of a transient conversation.

Git is important in this design for a specific reason. It does not merely store the final code. It records the

¹A secondary proposal-state mode can preserve constrained suggestions, discussion states, and approval status inside R when Git is not the primary review system. In this paper, proposal state is treated as supporting functionality rather than as the central contribution.

evolution of both design and implementation. If a coding assistant modifies a specification, renders a review page, and then changes code, reviewers can compare those changes as linked but distinct artifacts. This matters because multi-agent and asynchronous coding systems already use branch, isolation, merge, and executable verification patterns to preserve a coherent integrated state (Geng and Neubig 2026). `agentr` uses a lighter version of that discipline: it does not manage parallel engineering agents itself, but it makes the design artifacts that precede implementation visible to the same version-control practices that developers already use for code.

The result is a middle layer between prompt engineering and autonomous software development. Prompt engineering can influence a coding assistant, but it is weak at preserving approved task state over time. A full agent platform can execute workflows, but it may bind the project to a particular runtime or orchestration stack. `agentr` occupies the space between them. It gives the assistant a structured object to work on, gives the human reviewer a visible design to approve, and gives the repository a record of how both the design and implementation changed.

Contribution 3: visual review and staged autonomy as governance infrastructure

The third contribution is visual review. Structured files are useful only if humans can inspect them without reading every YAML field. `agentr` therefore renders specifications into review pages that show workflow decomposition, ordering, gates, memory fields, curated knowledge, implementation notes, warnings, and feedback fields. The function of this layer is not to make the workflow look polished. Its function is to make the review task tractable: reviewers should be able to see what the assistant is expected to do, what assumptions it is using, what remains unresolved, and where automation should stop.

This review layer addresses a recurring governance problem in human-AI work. Humans need enough visibility to exercise judgment, but raw logs and configuration files can be too detailed to support practical oversight. Research-object approaches to inspectable AI argue that AI-assisted research should be governed through structured provenance records, workflow artifacts, logs, and human-oversight documentation rather than through vague self-attestation (Binkyte et al. 2026). Repository-level review architectures similarly emphasize staged decomposition, artifact-first outputs, and repository-context construction as ways to improve auditability and reuse (Zhang 2026). The need for this layer is also supported by system-level accounts of agentic risk and accountability: harms can arise from interactions among model outputs, tools, memory, retrieval, APIs, and orchestration, so oversight requires workflow-level provenance and monitoring rather than isolated model explanations (Ghosh et al. 2025; Zhu et al. 2026). `agentr` applies this principle to coding-AI scaffolding: review should be attached to source specifications and implementation targets, not only to the final response or final patch.

Visual review also helps make autonomy a design variable. In agent workflows, autonomy should not be inferred directly from the model’s capability. It depends on how the system distributes control between the user and the agent: whether the user acts as operator, collaborator, consultant, approver, or observer (Feng, McDonald, and Zhang 2025). `agentr` therefore treats autonomy boundaries as part of the specification. A workflow step can be framed as requiring human review, deterministic execution, assistant proposal,

or downstream runtime implementation. This matters because the same model action can carry different risk depending on whether it is a suggestion, an editable draft, an approved code change, or an automatic deployment.

The limitation is equally important. A diagram is not evidence that a workflow is correct. A review page can create false confidence if it is stale, oversimplified, or disconnected from implementation. `agentr` addresses this limitation by keeping review pages generated from source specifications, exposing warnings and validation results, preserving file paths and implementation notes, and encouraging reviewers to compare specification changes with code changes in Git. Visual review is therefore an inspectability aid, not a substitute for testing, code review, or domain judgment.

Package boundary and relation to adjacent systems

The package boundary is deliberately modest. `agentr` supplies specification, scaffolding, validation, rendering, proposal support, and review. It does not replace model providers, agent runtimes, execution sandboxes, communication protocols, secure deployment systems, or domain-specific evaluation frameworks. Those components can be connected downstream, but they are not the package’s core contribution.

This boundary is a strength when the goal is coding-AI adoption in ordinary repositories. A framework that tries to solve orchestration, execution, communication, memory, governance, and evaluation at once can become too heavy for users who mainly need to make AI-assisted development inspectable. Conversely, a chat-only workflow is too thin when the task requires reviewable assumptions, reusable knowledge, implementation gates, and provenance. `agentr` selects the middle layer: it organizes the design and review objects that surround coding-AI work, while leaving execution and model choice open.

The same boundary also defines where `agentr` should not be overclaimed. It does not guarantee that a coding assistant will follow instructions. It does not prove that an implementation is correct. It does not determine whether an autonomous runtime is safe. It does not replace empirical evaluation of agent performance. Its value is narrower but still important: it gives developers a practical way to externalize and govern the intermediate structure of AI-assisted coding work.

Suitable users, tasks, and fields

`agentr` is most useful for developers, analysts, researchers, and small technical teams who already work with code repositories, R projects, scripts, data folders, Quarto documents, or computational notebooks. These users want the productivity of coding AIs, but they also need an inspectable path from goal to output. This user profile is consistent with multifaceted human-centered AI accounts that treat AI-supported work as an iterative workflow involving multiple roles, tools, artifacts, provenance records, and domain-specific interaction semantics rather than as a single human-tool exchange ([Rahman et al. 2023](#)).

The best tasks are multi-step, repeatable, and decomposable. They typically include data ingestion, transformation, interpretation, charting, report writing, validation, implementation, or human review. They benefit from explicit workflow nodes, visible gates, reusable memory, curated knowledge, documented assumptions,

rendered review pages, and version-controlled evolution. They are also tasks where coding assistance can accelerate work but should not be treated as a black box.

Good application scenarios include:

- data-analysis and reporting pipelines;
- chart and dashboard generation;
- research-assistant workflows;
- computational social science and economic analysis;
- financial or policy analysis where assumptions and exceptions matter;
- package maintenance and repository-level code revision;
- technical publishing with Quarto or R Markdown;
- internal business automation where review and provenance are required.

`agentr` is less useful for one-off casual chats, simple prompt templates, real-time conversational products, or applications that mainly need a provider-specific API client. It is also not meant to be the communication backend, the messaging layer, or the autonomous execution engine. Those concerns can live in adjacent packages or downstream systems.

Conclusion

`agentr` offers a practical answer to a specific problem: how to rely on external coding AIs without losing structural control, transparency, provenance, or human authority. It does so by turning intermediate design into inspectable files and review artifacts. The result is a middle layer between manual development and opaque automation.

The package’s contribution is concentrated in three places. Its task-local specifications turn workflow design, memory, knowledge, interface expectations, and autonomy boundaries into portable project artifacts. Its coding-assistant scaffolding pattern lets external AIs infer, revise, and implement against those artifacts while Git records the evolution of both design and code. Its visual review layer helps humans inspect assumptions, gates, warnings, and readiness before allowing downstream action.

The central claim is therefore modest but important. As coding AIs become more capable, developers need better scaffolding around them. `agentr` provides that scaffolding for R-centered, repository-based, review-sensitive workflows: not by replacing coding assistants or agent runtimes, but by making the work they perform more structured, inspectable, and governable.

References

- Amini, Soufiane, Yassine Benajiba, Cesare Bernardis, Paul Cayet, Hassan Chafi, Abderrahim Fathan, Louis Faucon, et al. 2025. “Open Agent Specification (Agent Spec): A Unified Representation for AI Agents.” <https://arxiv.org/abs/2510.04173v4>.
- Bansod, Prashik Buddhaghosh. 2025. “Distinguishing Autonomous AI Agents from Collaborative Agentic Systems: A Comprehensive Framework for Understanding Modern Intelligent Architectures.” <https://arxiv.org/abs/2510.04173v4>.

- <https://arxiv.org/abs/2506.01438v2>.
- Binkyte, Ruta, Sharif Abuaddba, Chamikara Mahawaga, Ming Ding, Natasha Fernandes, and Mario Fritz. 2026. "Inspectable AI for Science: A Research Object Approach to Generative AI Governance." <https://arxiv.org/abs/2604.11261v1>.
- Borghoff, Uwe M., Mark Minas, and Jannis Schopp. 2025. "Learning to Code with Context: A Study-Based Approach." <https://arxiv.org/abs/2512.05242v1>.
- Cao, Sheng, Zhao Chang, Chang Li, Hannan Li, Liyao Fu, and Ji Tang. 2026. "The Auton Agentic AI Framework." <https://arxiv.org/abs/2602.23720v1>.
- Du, Pengfei. 2026. "Memory for Autonomous LLM Agents: mechanisms, Evaluation, and Emerging Frontiers." <https://arxiv.org/abs/2603.07670v1>.
- Feng, K. J. Kevin, David W. McDonald, and Amy X. Zhang. 2025. "Levels of Autonomy for AI Agents." <https://arxiv.org/abs/2506.12469v2>.
- Geng, Jiayi, and Graham Neubig. 2026. "Effective Strategies for Asynchronous Software Engineering Agents." <https://arxiv.org/abs/2603.21489v1>.
- Ghosh, Shaona, Barnaby Simkin, Kyriacos Shiarlis, Soumili Nandi, Dan Zhao, Matthew Fiedler, Julia Bazinska, et al. 2025. "A Safety and Security Framework for Real-World Agentic Systems." <https://arxiv.org/abs/2511.21990v1>.
- Rahman, Sajjadur, Hannah Kim, Dan Zhang, Estevam Hruschka, and Eser Kandogan. 2023. "Towards Multifaceted Human-Centered AI." <https://arxiv.org/abs/2301.03656v1>.
- Sapkota, Ranjan, Konstantinos I. Roumeliotis, and Manoj Karkee. 2025. "AI Agents Vs. Agentic AI: A Conceptual Taxonomy, Applications and Challenges." <https://doi.org/10.1016/j.inffus.2025.103599>.
- Sergeyuk, Agnia, Yaroslav Golubev, Timofey Bryksin, and Iftekhar Ahmed. 2024. "Using AI-Based Coding Assistants in Practice: State of Affairs, Perceptions, and Ways Forward." <https://doi.org/10.1016/j.infsof.2024.107610>.
- Takerngsaksiri, Wannita, Jirat Pasuksmit, Patanamon Thongtanunam, Chakkrit Tantithamthavorn, Ruixiong Zhang, Fan Jiang, Jing Li, Evan Cook, Kun Chen, and Ming Wu. 2024. "Human-in-the-Loop Software Development Agents." <https://arxiv.org/abs/2411.12924v2>.
- Uulu, Choro Ulan, Mikhail Kulyabin, Iris Fuhrmann, Jan Joosten, Nuno Miguel Martins Pacheco, Filippos Petridis, Rebecca Johnson, Jan Bosch, and Helena Holmström Olsson. 2026. "How to Build AI Agents by Augmenting LLMs with Codified Human Expert Domain Knowledge? A Software Engineering Framework." <https://arxiv.org/abs/2601.15153v1>.
- Wang, Wei, Huilong Ning, Gaowei Zhang, Libo Liu, and Yi Wang. 2024. "Rocks Coding, Not Development—a Human-Centric, Experimental Evaluation of LLM-Supported SE Tasks." <https://arxiv.org/abs/2402.05650v3>.
- Wang, Zora Zhiruo, Yijia Shao, Omar Shaikh, Daniel Fried, Graham Neubig, and Diyi Yang. 2025. "How Do AI Agents Do Human Work? Comparing AI and Human Workflows Across Diverse Occupations." <https://arxiv.org/abs/2510.22780v2>.
- Wissuchek, Christopher, and Patrick Zschech. 2025. "Exploring Agentic Artificial Intelligence Systems: Towards a Typological Framework." <https://arxiv.org/abs/2508.00844v1>.
- Yang, Yingxuan, Mulei Ma, Yuxuan Huang, Huacan Chai, Chenyu Gong, Haoran Geng, Yuanjian Zhou, et al. 2025. "Agentic Web: Weaving the Next Web with AI Agents." <https://arxiv.org/abs/2507.21206v1>.

- Zhang, Peng. 2026. “RepoReviewer: A Local-First Multi-Agent Architecture for Repository-Level Code Review.” <https://arxiv.org/abs/2603.16107v1>.
- Zhu, Judy, Dhari Gandhi, Himanshu Joshi, Ahmad Rezaie Mianroodi, Sedef Akinli Kocak, and Dhanesh Ramachandran. 2026. “Interpreting Agentic Systems: Beyond Model Explanations to System-Level Accountability.” <https://arxiv.org/abs/2601.17168v1>.